```
    def __init__(self, <otherParameters>):
        # More code here
```

Because this class variable is owned by the class, in methods of the class you would refer to it as `MyDemoClass.myClassVariable`. Every object instantiated from a class has access to all class variables defined in the class.

There are two typical uses for class variables: defining a constant and creating a counter.

Class Variable Constants

You can create a class variable to be used as a constant, like so:

```
class MyClass():
    DEGREES_IN_CIRCLE = 360  # creating a class variable constant
```

To access this constant in methods of the class, you would write `MyClass.DEGREES_IN_CIRCLE`.

As a reminder, Python does not actually have constants. Instead, there is a convention among Python programmers that any variable whose name consists of all uppercase letters, with words separated by underscores, is meant to be treated as a constant. That is, this type of variable should never be reassigned.

We can also use class variable constants to save on resources (memory and time). Imagine we're writing a game where we create many instances of a `SpaceShip` class. We create a picture of a spaceship and place the file in a folder named *images*. Before considering class variables, the `__init__()` method of our `SpaceShip` class would start by instantiating an `Image` object like this:

```
class SpaceShip():
    def __init__(self, window, ...):
        self.image = pygwidgets.Image(window, (0, 0),
                                      'images/ship.png')
```